

0 poste

# DevOps Foundations

## Introduction

---

Vous êtes recruté·e comme DevOps Engineer Junior chez CloudNative Labs, une entreprise spécialisée dans l'accompagnement des startups vers des architectures cloud-native. Votre première mission : démontrer votre maîtrise des fondamentaux DevOps en construisant une infrastructure de référence.

Le CTO vous challenge : *"Je veux voir une stack de production, pas un docker-compose de tutoriel. Montre-moi que tu comprends les enjeux de scalabilité et de sécurité. Et surtout, je veux que tu crées tes propres services c'est comme ça qu'on apprend vraiment."*

Vous devez livrer une infrastructure conteneurisée complète comprenant :

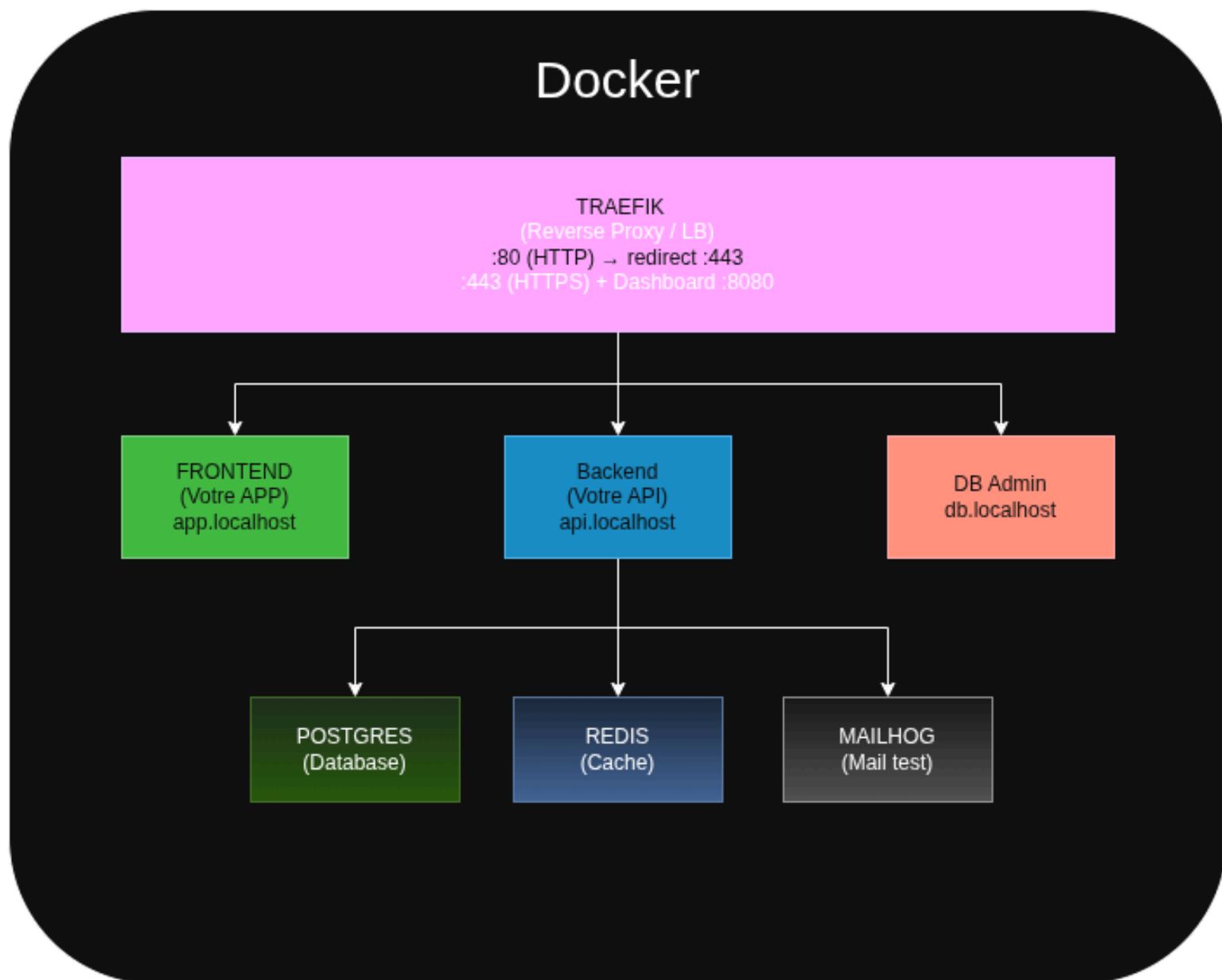
- Des microservices que vous développez vous-même (simples mais fonctionnels)
- Un reverse proxy intelligent gérant le routage et les certificats SSL
- Une architecture multi-environnements (dev, prod)
- Un workflow Git professionnel

 **Exigence CTO : Aucun port applicatif ne doit être exposé directement. Tout le trafic passe par le reverse proxy. Les credentials ne doivent JAMAIS apparaître en clair dans le code versionné.**



## Architecture Cible

L'infrastructure doit respecter le schéma suivant :





# Services à développer

---

💡 Philosophie : L'objectif n'est PAS de développer une application complexe, mais de créer des services simples qui démontrent votre maîtrise de la conteneurisation. Le code applicatif doit rester minimal c'est l'infrastructure qui compte.

## Service Backend (API)

Une API REST minimaliste [Node.js](#) qui doit :

- Route **GET /health** : Retourne `{"status": "ok", "service": "backend"}`
- Route **GET /** : Retourne un message de bienvenue avec la version
- Route **GET /db** : Teste la connexion PostgreSQL et retourne le statut
- Route **GET /cache** : Teste la connexion Redis (incrémente un compteur de visites)
- Route **POST /contact** : Envoie un email de test via MailHog

*Exemple de réponse /db :*

```
{  
  "status": "connected",  
  "database": "devops_db",  
  "timestamp": "2025-01-09T10:30:00Z"  
}
```

## Service Frontend

Une page web simple (HTML/CSS/JS ou framework léger) qui doit :

- Afficher un titre "DevOps Foundations – Dashboard".
- Afficher le statut des services en appelant les routes de l'API :
  -  Backend : OK /  Backend : DOWN
  -  Database : OK /  Database : DOWN
  -  Cache : OK /  Cache : DOWN



- Afficher le compteur de visites (depuis Redis)
- Un formulaire de contact simple (qui envoie vers [/contact](#))

 **Note :** Vous pouvez utiliser un simple fichier HTML servi par Nginx, ou un framework (React, Vue, Svelte...). Le choix technique est libre tant que le service est conteneurisé correctement.

## Travail attendu

---

### Partie 1 – Workflow Git & Culture DevOps

Livrable 1.1 : Workflow Git professionnel

- Implémentation stricte de GitFlow avec :
  - Branches protégées ([main](#), [develop](#))
  - Minimum 5 commits par branche feature (commits atomiques et bien nommés)
  - Utilisation de Conventional Commits ([feat](#);, [fix](#);, [docs](#);, [chore](#);)
  - Au moins un merge avec résolution de conflit documentée
- Fichier [CONTRIBUTING.md](#) détaillant :
  - La stratégie de branching
  - Les conventions de commits
  - Le processus de code review (self-review checklist)

Livrable 1.2 : Documentation Merge vs Rebase (1-2 pages)

- Analyse comparative avec exemples concrets
- Démonstration des deux approches dans l'historique Git du projet
- Politique choisie pour le projet avec justification technique

### Partie 2 – Développement & Conteneurisation

Livrable 2.1 : Services applicatifs

- Code source du backend avec les 5 routes requises
- Code source du frontend avec le dashboard de statut
- Les deux services doivent être fonctionnels et testables



## Livrable 2.2 : Dockerfiles optimisés

Pour chaque service (frontend, backend) :

- Multi-stage build obligatoire avec au minimum :
  - Stage **builder** : installation dépendances / compilation
  - Stage **production** : image minimale (alpine ou distroless)
- Bonnes pratiques de sécurité :
  - Utilisateur non-root avec UID/GID explicites
  - Aucun secret dans l'image (utilisation de variables d'environnement)
- Optimisation :
  - Ordre des instructions optimisé pour le cache
  - **.dockerignore** complet
  - Comparatif taille image avant/après optimisation (dans la doc)
- Labels OCI (**org.opencontainers.image.\***)
- Health checks intégrés

## Livrable 2.3 : Docker Compose multi-environnements

- Architecture de fichiers :
  - **docker-compose.yml** : configuration de base
  - **docker-compose.override.yml** : développement (hot-reload, volumes bind)
  - **docker-compose.prod.yml** : production (replicas, limits, restart policies)
- Configuration obligatoire pour chaque service :
  - Health checks avec intervalles appropriés
  - Restart policies adaptées
  - Ressources limitées en prod (**deploy.resources.limits**)
  - Logging driver configuré
- Réseaux isolés :
  - Réseau **frontend** : Traefik + services exposés
  - Réseau **backend** : services internes (DB, Redis, MailHog)
  - Isolation stricte : la DB n'est PAS sur le réseau frontend



- Volumes nommés pour la persistance (PostgreSQL, Redis)
- Gestion des secrets :
  - Fichier `.env.example` documenté
  - `.env` dans `.gitignore`

### Partie 3 – Reverse Proxy Traefik

#### Livrable 3.1 : Configuration Traefik

- Traefik v3 avec :
  - Docker provider pour la découverte dynamique
  - Configuration statique (`traefik.yml`)
- Entrypoints :
  - `web` (:80) avec redirection automatique vers HTTPS
  - `websecure` (:443) avec certificats auto-générés (mkcert)
- Routage dynamique via labels Docker :
  - `app.localhost` → Frontend
  - `api.localhost` → Backend
  - `db.localhost` → Adminer (administration DB)
  - `mail.localhost` → MailHog (interface web)
  - `traefik.localhost` → Dashboard Traefik
- Middlewares obligatoires :
  - Rate limiting sur l'API (ex: 100 req/min)
  - Headers de sécurité (HSTS, X-Frame-Options, X-Content-Type-Options)
  - Basic Auth sur le dashboard Traefik et Adminer
  - Compression gzip
- Load balancing : au moins 2 replicas du backend avec démonstration

#### Livrable 3.2 : Documentation réseau

##### Schéma détaillé des flux réseau

- Explication du fonctionnement de Traefik (providers, routers, services, middlewares)
- Justification des choix de sécurité



## Partie 4 – Documentation & Démonstration

Livrable 4.1 : README complet

- Prérequis (Docker, mkcert, etc.)
- Guide d'installation pas à pas
- Commandes utiles (démarrage, arrêt, logs, rebuild)
- URLs des services

Livrable 4.2 : Screencast de démonstration (5-10 min)

- Clonage du dépôt et configuration initiale
- Lancement avec `docker compose up`
- Navigation sur tous les services via leurs URLs
- Démonstration du load balancing (2 replicas backend)
- Arrêt/relance avec vérification de la persistance des données



## Aller plus loin

---

Pour les plus motivés : Ces fonctionnalités avancées démontrent une vraie maturité DevOps et seront valorisées.

### Bonus 1 : Stack Monitoring

- Prometheus configuré pour scraper :
  - Métriques Traefik
  - Métriques des conteneurs via cAdvisor
- Grafana avec :
  - Datasource Prometheus pré-configurée
  - Au moins 2 dashboards (Traefik + Docker)
  - Accessible via [monitor.localhost](http://monitor.localhost)

### Bonus 2 : Orchestration Docker Swarm

- Fichier [docker-stack.yml](#) avec :
  - Replicas pour haute disponibilité
  - Rolling updates configurés
  - Rollback automatique en cas d'échec
- Démonstration d'un rolling update sans downtime

### Bonus 3 : Watchtower

- Mises à jour automatiques des conteneurs
- Notifications configurées (webhook, email, ou Slack)
- Labels pour inclure/exclure des conteneurs

### Bonus 4 : Portainer

- Interface de gestion déployée
- Accessible via Traefik ([portainer.localhost](http://portainer.localhost))

### Bonus 5 : Sécurité avancée

- Scan d'images avec Trivy ou Docker Scout
- Rapport de vulnérabilités dans la documentation





# Compétences visées

---

## **Concevoir et déployer des infrastructures DevOps automatisées**

- Analyser l'existant et qualifier les demandes en réponse aux demandes internes ou externes
- Déployer et orchestrer des environnements conteneurisés
- Assurer la portabilité, la scalabilité et la résilience des applications en optimisant la consommation des ressources

## **Concevoir et développer une application informatique**

- Recommander un environnement informatique éco-responsable en garantissant la cohérence du système
- Développer des applications métiers sécurisées en optimisant l'efficacité des développeurs
- Concevoir une architecture applicative en analysant les besoins fonctionnels et les exigences de sécurité

## **Piloter des projets informatiques**

- Communication technique et non-technique
- Développer et intégrer des stratégies de mitigation des risques en identifiant et évaluant les menaces potentielles afin de gérer la sécurité du projet

## **Définir une stratégie de systèmes d'information**

- Comparer et schématiser les architectures selon leurs caractéristiques et cas d'usage
- Comprendre les avantages et inconvénients de chaque type d'architecture afin de recommander les solutions architecturales les plus appropriées



# Rendu

---

## Rendu

Le projet est à rendre sur votre dépôt GitHub  
<https://github.com/prenom-nom/devops-foundations>

Structure attendue du dépôt :

```
devops-foundations/
├── README.md           # Guide complet
├── CONTRIBUTING.md     # Workflow Git
├── .gitignore
├── .env.example
├── src/
│   ├── frontend/      # Votre code frontend
│   │   ├── Dockerfile
│   │   ├── .dockerignore
│   │   └── ...
│   └── backend/       # Votre code backend
│       ├── Dockerfile
│       ├── .dockerignore
│       └── ...
├── traefik/
│   ├── traefik.yml
│   └── dynamic/
│       └── middlewares.yml
├── docker-compose.yml
├── docker-compose.override.yml
├── docker-compose.prod.yml
├── docs/
│   ├── merge-vs-rebase.md
│   ├── architecture-reseau.md
│   └── images/
│       └── schema-architecture.png
├── scripts/
│   ├── init.sh
│   └── generate-certs.sh
```

**Modalité : Projet individuel**



# Base de connaissances

---

## Git & Workflow

- [Atlassian - GitFlow Workflow](#)
- [Conventional Commits \(FR\)](#)
- [Pro Git Book \(Français\)](#)

## Docker

- [Dockerfile Best Practices](#)
- [Multi-stage builds](#)
- [Docker Compose Specification](#)

## Traefik

- [Documentation Officielle](#)
- [Docker Provider](#)
- [Middlewares](#)

## Développement rapide (exemples)

- [Express.js \(Node.js\)](#)
- [FastAPI \(Python\)](#)
- [Gin \(Go\)](#)

## Outils bonus

- [Prometheus](#)
- [Grafana](#)
- [Watchtower](#)
- [Portainer](#)
- [Docker Swarm](#)

## Outils utilitaires (gratuits)

- [Docker Desktop](#)
- [mkcert](#) - Certificats SSL locaux
- [Draw.io](#) - Schémas
- [Dive](#) - Analyse layers Docker